

DinoFracture

Generated by Doxygen 1.9.4

1 DinoFracture	1
1.1 About DinoFracture	1
1.2 Basic Usage	1
1.3 Support	1
2 Namespace Index	3
2.1 Package List	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 Namespace Documentation	9
5.1 DinoFracture Namespace Reference	9
5.1.1 Enumeration Type Documentation	11
5.1.1.1 FracturedMeshResultFlags	11
5.1.1.2 FractureIssueResolution	11
5.1.1.3 FractureType	11
5.1.1.4 FractureUVScale	12
5.1.1.5 MeshTopologyError	12
5.1.1.6 MeshValidity	12
5.2 DinoFracture.Internal Namespace Reference	13
6 Class Documentation	15
6.1 DinoFracture.AsyncFractureOperation Class Reference	15
6.1.1 Detailed Description	16
6.1.2 Member Function Documentation	16
6.1.2.1 Wait()	16
6.2 DinoFracture.AsyncFractureResult Class Reference	16
6.2.1 Detailed Description	17
6.3 DinoFracture.AsyncShatterOperation Class Reference	17
6.3.1 Detailed Description	17
6.4 DinoFracture.AsyncSliceOperation Class Reference	18
6.4.1 Detailed Description	18
6.5 DinoFracture.CleanupMeshOnDestroy Class Reference	19
6.5.1 Detailed Description	19
6.6 DinoFracture.Internal.DestroyOnAudioFinish Class Reference	19
6.6.1 Detailed Description	19
6.7 DinoFracture.DisableObjectsOnFracture Class Reference	19
6.7.1 Detailed Description	19
6.8 DinoFracture.EdgeError Struct Reference	19
6.8.1 Detailed Description	20

6.9 DinoFracture.FractureDetails Class Reference	20
6.9.1 Detailed Description	21
6.9.2 Member Function Documentation	21
6.9.2.1 IsValid()	21
6.9.3 Member Data Documentation	21
6.9.3.1 InsideMaterialIndex	22
6.9.3.2 IssueResolution	22
6.9.3.3 SeparateDisjointPieces	22
6.10 DinoFracture.FracturedMesh Struct Reference	22
6.10.1 Detailed Description	22
6.11 DinoFracture.FractureEngine Class Reference	23
6.11.1 Detailed Description	24
6.11.2 Member Function Documentation	24
6.11.2.1 StartFracture()	24
6.11.3 Property Documentation	24
6.11.3.1 MaxRunningFractures	25
6.12 DinoFracture.FractureEngineBase Class Reference	25
6.12.1 Detailed Description	26
6.12.2 Member Function Documentation	26
6.12.2.1 ClearCachedFractureData()	26
6.12.2.2 RunOnMainThread()	26
6.12.3 Property Documentation	26
6.12.3.1 ForceSynchronousPreFractureInEditor	26
6.13 DinoFracture.FractureGeometry Class Reference	27
6.13.1 Detailed Description	29
6.13.2 Member Function Documentation	29
6.13.2.1 CreateSlicePlane()	29
6.13.2.2 ForceValidGeometry()	30
6.13.2.3 Fracture() [1/2]	30
6.13.2.4 Fracture() [2/2]	30
6.13.2.5 FractureAndForget() [1/2]	30
6.13.2.6 FractureAndForget() [2/2]	31
6.13.3 Member Data Documentation	31
6.13.3.1 EvenlySizedPieces	31
6.13.3.2 NumGenerations	31
6.13.3.3 NumIterations	31
6.13.3.4 OptimizeMaterialUsage	32
6.13.3.5 SeparateDisjointPieces	32
6.13.3.6 SlicePlanes	32
6.14 DinoFracture.FractureOnClick Class Reference	32
6.14.1 Detailed Description	32
6.15 DinoFracture.FractureOnCollision Class Reference	32

6.15.1 Detailed Description	33
6.16 DinoFracture.FractureOnInput Class Reference	33
6.16.1 Detailed Description	33
6.17 DinoFracture.FractureResult Class Reference	33
6.17.1 Detailed Description	33
6.17.2 Member Function Documentation	34
6.17.2.1 GetMeshes()	34
6.18 DinoFracture.GlueEdgeOnFracture Class Reference	34
6.18.1 Detailed Description	34
6.19 DinoFracture.NotifyOnFracture Class Reference	34
6.19.1 Detailed Description	35
6.20 DinoFracture.FractureGeometry.OnFractureEvent Class Reference	35
6.20.1 Detailed Description	35
6.21 DinoFracture.OnFractureEventArgs Class Reference	35
6.21.1 Detailed Description	36
6.21.2 Member Function Documentation	36
6.21.2.1 GetMeshes()	36
6.22 DinoFracture.PlaySoundOnFracture Class Reference	36
6.22.1 Detailed Description	36
6.23 DinoFracture.PreFracturedGeometry Class Reference	36
6.23.1 Detailed Description	37
6.23.2 Member Data Documentation	37
6.23.2.1 GeneratedPieces	37
6.24 DinoFracture.RuntimeFracturedGeometry Class Reference	38
6.24.1 Detailed Description	38
6.24.2 Member Data Documentation	39
6.24.2.1 Asynchronous	39
6.25 DinoFracture.ShatterDetails Class Reference	39
6.25.1 Detailed Description	40
6.25.2 Member Function Documentation	40
6.25.2.1 IsValid()	40
6.25.3 Member Data Documentation	41
6.25.3.1 EvenlySizedPieces	41
6.26 DinoFracture.SliceDetails Class Reference	41
6.26.1 Detailed Description	42
6.26.2 Member Function Documentation	42
6.26.2.1 IsValid()	42
6.27 DinoFracture.SlicePlane Struct Reference	42
6.27.1 Detailed Description	43
6.27.2 Member Data Documentation	43
6.27.2.1 Rotation	43
6.27.2.2 Scale	43

6.28 DinoFracture.FractureGeometry.SlicePlaneSerializable Struct Reference	44
6.28.1 Detailed Description	44
6.28.2 Member Function Documentation	44
6.28.2.1 ToSlicePlane()	44
6.29 DinoFracture.TransferJointsOnFracture Class Reference	45
6.29.1 Detailed Description	45
6.30 DinoFracture.TriggerExplosionOnCollision Class Reference	45
6.30.1 Detailed Description	45
Index	47

Chapter 1

DinoFracture

1.1 About DinoFracture

DinoFracture is a tool that can shatter meshes either in real time or in the editor. It works on any mesh, skinned or static, but works best with water-tight meshes. A water-tight mesh is one that does not have holes exposing the backfaces of the mesh polygons. Well-formed holes, like in donuts, *are* allowed.

1.2 Basic Usage

Scripts are located in the **DinoFracture\Plugin\Scripts** directory. There are three kinds of scripts in the folder:

- Trigger scripts that cause fractures.
- Scripts that allow this object to be fractured
- Notify scripts that perform an action when fractured
- Engine scripts that are not meant to be used by the user The general flow is to apply either the [PreFracturedGeometry](#) or [RuntimeFracturedGeometry](#) to an object with a mesh. Apply a trigger script to either the object being fractured (ex: FractureOnCollision) or an external object that will cause a fracture. Apply notify scripts (ex: PlaySoundOnFracture) to perform actions when the fracture occurs. See the **Playground** scene in **DinoFracture\Demo** for an example of some different usage patterns.

1.3 Support

If you have any questions about how the plugin works or run into any issues, please contact us at **support@entropysoftware.io**

Chapter 2

Namespace Index

2.1 Package List

Here are the packages with brief descriptions (if available):

DinoFracture	9
DinoFracture.Internal	13

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

DinoFracture.AsyncFractureOperation	15
DinoFracture.AsyncShatterOperation	17
DinoFracture.AsyncSliceOperation	18
DinoFracture.AsyncFractureResult	16
DinoFracture.CleanupMeshOnDestroy	19
DinoFracture.Internal.DestroyOnAudioFinish	19
DinoFracture.DisableObjectsOnFracture	19
DinoFracture.EdgeError	19
DinoFracture.FractureDetails	20
DinoFracture.ShatterDetails	39
DinoFracture.SliceDetails	41
DinoFracture.FracturedMesh	22
DinoFracture.FractureEngineBase	25
DinoFracture.FractureEngine	23
DinoFracture.FractureGeometry	27
DinoFracture.PreFracturedGeometry	36
DinoFracture.RuntimeFracturedGeometry	38
DinoFracture.FractureOnClick	32
DinoFracture.FractureOnCollision	32
DinoFracture.FractureOnInput	33
DinoFracture.FractureResult	33
DinoFracture.GlueEdgeOnFracture	34
DinoFracture.NotifyOnFracture	34
DinoFracture.FractureGeometry.OnFractureEvent	35
DinoFracture.OnFractureEventArgs	35
DinoFracture.PlaySoundOnFracture	36
DinoFracture.SlicePlane	42
DinoFracture.FractureGeometry.SlicePlaneSerializable	44
DinoFracture.TransferJointsOnFracture	45
DinoFracture.TriggerExplosionOnCollision	45

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

DinoFracture.AsyncFractureOperation	Tracks completion and returns the results of a fragmentation operation.	15
DinoFracture.AsyncFractureResult	The result of a fracture.	16
DinoFracture.AsyncShatterOperation	Tracks completion and returns the results of a shatter.	17
DinoFracture.AsyncSliceOperation	Tracks completion and returns the results of a slice.	18
DinoFracture.CleanupMeshOnDestroy	Applying this script to the fracture template will ensure that the generated fracture mesh will be cleaned up properly when the fracture piece is destroyed	19
DinoFracture.Internal.DestroyOnAudioFinish	This component is automatically added to temporary sound game objects created by the PlaySoundOnFracture component. It is not intended to be added by the user.	19
DinoFracture.DisableObjectsOnFracture	Adding this to the fracturing game object will allow other game objects to be turned off (set inactive) when this game object is fractured.	19
DinoFracture.EdgeError	Used to display geometry errors on the mesh in the editor	19
DinoFracture.FractureDetails	Basic information for any sort of mesh fragmentation.	20
DinoFracture.FracturedMesh	An individual fracture piece's geometry	22
DinoFracture.FractureEngine	This component is created on demand to manage the fracture coroutines. It is not intended to be added by the user.	23
DinoFracture.FractureEngineBase	Base class for the fracture engine.	25
DinoFracture.FractureGeometry	This is the base class for the PreFractureGeometry and RuntimeFractureGeometry components. As such, it is not intended to be directly added to any game object even though fracture initiator components rely on it.	27
DinoFracture.FractureOnClick	Casts a simple mouse ray on left click and calls Fracture() on the hit collider game object. . . .	32
DinoFracture.FractureOnCollision	This component will cause a fracture to happen at the point of impact.	32

DinoFracture.FractureOnInput	Apply this on the fracturing game object. When the specified key is pressed, the object will fracture.	33
DinoFracture.FractureResult	The result of a fracture.	33
DinoFracture.GlueEdgeOnFracture	If the fracture pieces intersects with a specified trigger when created, the rigid body is destroyed and the piece becomes static. Otherwise, the piece will turn on gravity. It's best used if the FractureTemplate's rigid body is set to not use gravity initially.	34
DinoFracture.NotifyOnFracture	When added to the same game object as the FractureGeometry , this script can be used to notify external game objects of this object's fracture completion. The external objects need a script with the "OnFracture" callback method.	34
DinoFracture.FractureGeometry.OnFractureEvent	OnFracture() Unity event wrapper	35
DinoFracture.OnFractureEventArgs	Argument passed to OnFracture message	35
DinoFracture.PlaySoundOnFracture	An object with this component will play the audio source when fractured.	36
DinoFracture.PreFracturedGeometry	Apply this component to any game object you wish to pre-fracture. Pre-fracturing is a way of baking fracture pieces into the scene. Each time the object is fractured, the same set of pieces will activate. This is very useful when creating a large number of pieces or high poly meshes, which would be too slow to create at runtime. The pieces will be in the scene as a disabled root object with piece children. When the object is fractured, those pieces will activate.	36
DinoFracture.RuntimeFracturedGeometry	Apply this component to any game object you wish to fracture while running in game mode. Runtime fractures will produce a unique set of pieces with each fracture. However, this is at the cost of computational time. It is recommended that both the piece count and poly count are kept low. This component is most effective when FractureRadius is set to a value in-between 0 and 1.	38
DinoFracture.ShatterDetails	Required information needed by the engine to produce a fracture	39
DinoFracture.SliceDetails	Required information needed by the engine to slice a mesh	41
DinoFracture.SlicePlane	Defines a plane that slices the mesh in half.	42
DinoFracture.FractureGeometry.SlicePlaneSerializable	Unity cannot handle the serializable attribute on types defined in dlls. So, we have to duplicate the SlicePlane structure here in order to save it.	44
DinoFracture.TransferJointsOnFracture	When this object is fractured, the joint component on the object will be copied to this piece if this piece is sufficiently close to the joint position. Without this component, joints are broken after fracturing.	45
DinoFracture.TriggerExplosionOnCollision	Triggers a fracture + explosion when this game object is collided with	45

Chapter 5

Namespace Documentation

5.1 DinoFracture Namespace Reference

Classes

- class [AsyncFractureOperation](#)
Tracks completion and returns the results of a fragmentation operation.
- class [AsyncFractureResult](#)
The result of a fracture.
- class [AsyncShatterOperation](#)
Tracks completion and returns the results of a shatter.
- class [AsyncSliceOperation](#)
Tracks completion and returns the results of a slice.
- class [CleanupMeshOnDestroy](#)
Applying this script to the fracture template will ensure that the generated fracture mesh will be cleaned up properly when the fracture piece is destroyed.
- class [DisableObjectsOnFracture](#)
Adding this to the fracturing game object will allow other game objects to be turned off (set inactive) when this game object is fractured.
- struct [EdgeError](#)
Used to display geometry errors on the mesh in the editor
- class [FractureDetails](#)
Basic information for any sort of mesh fragmentation.
- struct [FracturedMesh](#)
An individual fracture piece's geometry
- class [FractureEngine](#)
This component is created on demand to manage the fracture coroutines. It is not intended to be added by the user.
- class [FractureEngineBase](#)
Base class for the fracture engine.
- class [FractureGeometry](#)
This is the base class for the PreFractureGeometry and RuntimeFractureGeometry components. As such, it is not intended to be directly added to any game object even though fracture initiator components rely on it.
- class [FractureOnClick](#)
Casts a simple mouse ray on left click and calls Fracture() on the hit collider game object.
- class [FractureOnCollision](#)
This component will cause a fracture to happen at the point of impact.

- class [FractureOnInput](#)
Apply this on the fracturing game object. When the specified key is pressed, the object will fracture.
- class [FractureResult](#)
The result of a fracture.
- class [GlueEdgeOnFracture](#)
If the fracture pieces intersects with a specified trigger when created, the rigid body is destroyed and the piece becomes static. Otherwise, the piece will turn on gravity. It's best used if the FractureTemplate's rigid body is set to not use gravity initially.
- class [NotifyOnFracture](#)
When added to the same game object as the [FractureGeometry](#), this script can be used to notify external game objects of this object's fracture completion. The external objects need a script with the "OnFracture" callback method.
- class [OnFractureEventArgs](#)
Argument passed to OnFracture message
- class [PlaySoundOnFracture](#)
An object with this component will play the audio source when fractured.
- class [PreFracturedGeometry](#)
Apply this component to any game object you wish to pre-fracture. Pre-fracturing is a way of baking fracture pieces into the scene. Each time the object is fractured, the same set of pieces will activate. This is very useful when creating a large number of pieces or high poly meshes, which would be too slow to create at runtime. The pieces will be in the scene as a disabled root object with piece children. When the object is fractured, those pieces will activate.
- class [RuntimeFracturedGeometry](#)
Apply this component to any game object you wish to fracture while running in game mode. Runtime fractures will produce a unique set of pieces with each fracture. However, this is at the cost of computational time. It is recommended that both the piece count and poly count are kept low. This component is most effective when FractureRadius is set to a value in-between 0 and 1.
- class [ShatterDetails](#)
Required information needed by the engine to produce a fracture
- class [SliceDetails](#)
Required information needed by the engine to slice a mesh
- struct [SlicePlane](#)
Defines a plane that slices the mesh in half.
- class [TransferJointsOnFracture](#)
When this object is fractured, the joint component on the object will be copied to this piece if this piece is sufficiently close to the joint position. Without this component, joints are broken after fracturing.
- class [TriggerExplosionOnCollision](#)
Triggers a fracture + explosion when this game object is collided with.

Enumerations

- enum [FractureUVScale](#) { [EntireMesh](#) , [Piece](#) }
Algorithm used to generate UVs on inside faces
- enum [FractureIssueResolution](#) { [NoAction](#) , [DisableGameObject](#) , [ReplaceMeshCollider](#) }
Technique used to handle pieces that generated with potential issues.
- enum [MeshValidity](#) { [Unknown](#) , [Valid](#) , [NeedsCleaning](#) , [Unrecoverable](#) }
Denotes the state of the mesh's topology and readiness to be fractured.
- enum [MeshTopologyError](#) {
 [None](#) = 0 , [DegenerateTriangles](#) = 1 << 0 , [OpenFaces](#) = 1 << 1 , [CloseVertices](#) = 1 << 2 ,
 [FanFaces](#) = 1 << 3 }
Returns a list of errors in the mesh's topology.
- enum [FracturedMeshResultFlags](#) { [NoIssues](#) = 0 , [SmallVertexCount](#) = 1 << 0 }
Additional flags describing the generated fracture mesh.
- enum [FractureType](#) { [Shatter](#) , [Slice](#) }
The type of fracture to perform

5.1.1 Enumeration Type Documentation

5.1.1.1 FracturedMeshResultFlags

enum `DinoFracture.FracturedMeshResultFlags`

Additional flags describing the generated fracture mesh.

Enumerator

NoIssues	Everything is okay
SmallVertexCount	There is a substantially small number of vertices in this mesh. This can cause problems when generating a mesh collider.

5.1.1.2 FractureIssueResolution

enum `DinoFracture.FractureIssueResolution`

Technique used to handle pieces that generated with potential issues.

Enumerator

NoAction	Do nothing internally - let the user take action
DisableGameObject	Completely disable the game object
ReplaceMeshCollider	Replaces the mesh collider with a sphere collider if the mesh collider could potentially fail to generate.

5.1.1.3 FractureType

enum `DinoFracture.FractureType`

The type of fracture to perform

Enumerator

Shatter	Traditional fracture. Divide the mesh into many random sized pieces.
Slice	Use one or more user-defined planes to cut the mesh

5.1.1.4 FractureUVScale

enum `DinoFracture.FractureUVScale`

Algorithm used to generate UVs on inside faces

Enumerator

EntireMesh	The fracture UVs map to the size of the original mesh.
Piece	The fracture UVs map to the size of each individual piece.

5.1.1.5 MeshTopologyError

enum `DinoFracture.MeshTopologyError`

Returns a list of errors in the mesh's topology.

Enumerator

None	No errors. Mesh is valid.
DegenerateTriangles	The mesh has zero-area triangles (either 3 vertices in a line or collapsed to a single point) This can be caused by poor export from DCC tools or by Unity doing a bad triangulation job importing NGons from an FBX. Degenerate triangles can be fixed through automatic mesh cleaning.
OpenFaces	The mesh is not water-tight. Mathematically, this means there is at least one edge that has exactly one associated triangle. Normal edges are shared by exactly 2 triangles. Open faces can be fixed through automatic mesh cleaning as long as there are at 3 connected open edges that all belong a single plane. Multiple groups of open faces can be fixed for a single mesh.
CloseVertices	The mesh has vertices that are extremely close, but not exactly on the same position. These vertices will be merged before during the cleaning process.
FanFaces	An edge is being shared by 3 or more triangles. It is no longer possible to determine what is inside vs outside of the mesh. This type of error cannot be fixed automatically.

5.1.1.6 MeshValidity

enum `DinoFracture.MeshValidity`

Denotes the state of the mesh's topology and readiness to be fractured.

Enumerator

Unknown	Meshes should only be in this state when the fracture script is first created and has yet to be clicked on in the editor. The mesh will be first checked and cleaned before fracturing.
Valid	The mesh is known to be valid and no checks or cleaning will be performed.

Enumerator

NeedsCleaning	The mesh is known to be invalid for fracturing. The mesh will be cleaned internally before starting the fracture.
Unrecoverable	The mesh is beyond what can be done through automatic cleaning. Fracturing can still happen, but there is a high chance that random slices will result in a failed fracture.

5.2 DinoFracture.Internal Namespace Reference

Classes

- class [DestroyOnAudioFinish](#)

This component is automatically added to temporary sound game objects created by the [PlaySoundOnFracture](#) component. It is not intended to be added by the user.

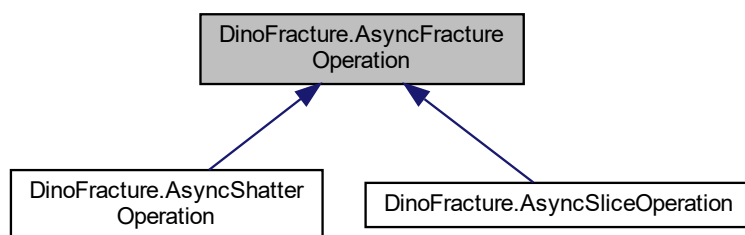
Chapter 6

Class Documentation

6.1 DinoFracture.AsyncFractureOperation Class Reference

Tracks completion and returns the results of a fragmentation operation.

Inheritance diagram for DinoFracture.AsyncFractureOperation:



Public Member Functions

- void `Wait` (int msTimeout)
Wait on the fracture to complete

Protected Member Functions

- void `SetComplete` ()
Marks this operation as completed

Properties

- [FractureDetails](#) **Details** [get, set]
The original details passed into the operation
- [FractureResult](#) **Result** [get, protected set]
The result of the fracture. This is not set until IsComplete is true.
- bool **IsComplete** [get]
True if the fracture has completed, false otherwise. If this is a synchronous fracture, this value will always be true by the end of the fracture call.

6.1.1 Detailed Description

Tracks completion and returns the results of a fragmentation operation.

6.1.2 Member Function Documentation

6.1.2.1 Wait()

```
void DinoFracture.AsyncFractureOperation.Wait (
    int msTimeout )
```

Wait on the fracture to complete

Parameters

<i>msTimeout</i>	Max time to wait. 0 to not wait, -1 to wait forever.
------------------	--

6.2 DinoFracture.AsyncFractureResult Class Reference

The result of a fracture.

Properties

- bool **IsComplete** [get]
Returns true if the operation has finished; false otherwise. This value will always be true for synchronous fractures.
- bool **IsSuccessful** [get]
Returns true if the operation has finished and returned valid results.
- [FractureGeometry](#) **FractureGeometry** [get]
The original script that initiated the fracture
- GameObject **PiecesRoot** [get]
The root of the pieces of the resulting fracture
- Bounds **EntireMeshBounds** [get]
The bounds of the original mesh

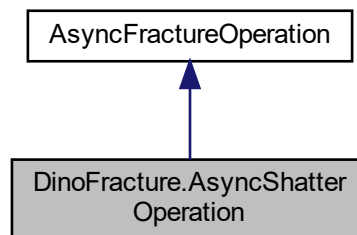
6.2.1 Detailed Description

The result of a fracture.

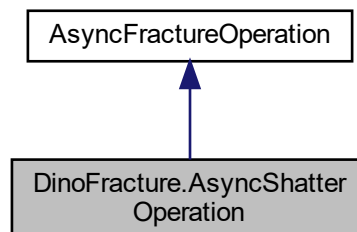
6.3 DinoFracture.AsyncShatterOperation Class Reference

Tracks completion and returns the results of a shatter.

Inheritance diagram for DinoFracture.AsyncShatterOperation:



Collaboration diagram for DinoFracture.AsyncShatterOperation:



Properties

- new [ShatterDetails](#) **Details** [get, set]
The original details passed into the operation

Additional Inherited Members

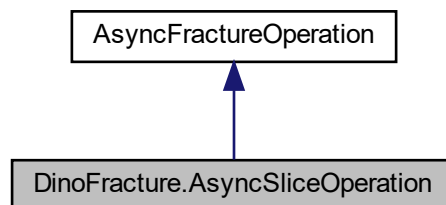
6.3.1 Detailed Description

Tracks completion and returns the results of a shatter.

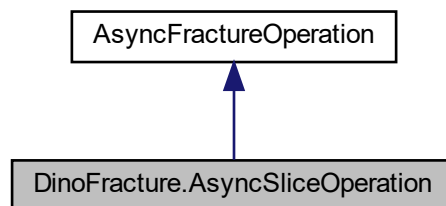
6.4 DinoFracture.AsyncSliceOperation Class Reference

Tracks completion and returns the results of a slice.

Inheritance diagram for DinoFracture.AsyncSliceOperation:



Collaboration diagram for DinoFracture.AsyncSliceOperation:



Properties

- new [SliceDetails](#) **Details** [get, set]
The original details passed into the operation

Additional Inherited Members

6.4.1 Detailed Description

Tracks completion and returns the results of a slice.

6.5 DinoFracture.CleanupMeshOnDestroy Class Reference

Applying this script to the fracture template will ensure that the generated fracture mesh will be cleaned up properly when the fracture piece is destroyed.

Inherits MonoBehaviour.

6.5.1 Detailed Description

Applying this script to the fracture template will ensure that the generated fracture mesh will be cleaned up properly when the fracture piece is destroyed.

It is always a good idea to add this to the fracture template.

6.6 DinoFracture.Internal.DestroyOnAudioFinish Class Reference

This component is automatically added to temporary sound game objects created by the [PlaySoundOnFracture](#) component. It is not intended to be added by the user.

Inherits MonoBehaviour.

6.6.1 Detailed Description

This component is automatically added to temporary sound game objects created by the [PlaySoundOnFracture](#) component. It is not intended to be added by the user.

6.7 DinoFracture.DisableObjectsOnFracture Class Reference

Adding this to the fracturing game object will allow other game objects to be turned off (set inactive) when this game object is fractured.

Inherits MonoBehaviour.

6.7.1 Detailed Description

Adding this to the fracturing game object will allow other game objects to be turned off (set inactive) when this game object is fractured.

6.8 DinoFracture.EdgeError Struct Reference

Used to display geometry errors on the mesh in the editor

Public Attributes

- readonly Vector3 **V0**
First edge vertex, in object space
- readonly Vector3 **V1**
Second edge vertex, in object space
- readonly [MeshTopologyError](#) **Errors**
Error(s) associated with this edge. Useful for coloring the edges.
- readonly UnityEngine.GameObject **GameObject**
The game object associated with the errors

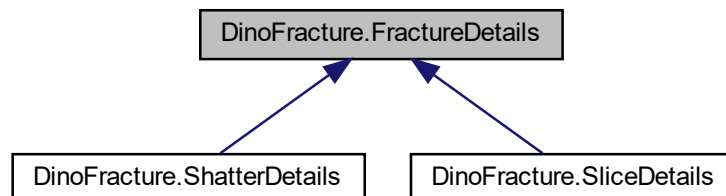
6.8.1 Detailed Description

Used to display geometry errors on the mesh in the editor

6.9 DinoFracture.FractureDetails Class Reference

Basic information for any sort of mesh fragmentation.

Inheritance diagram for DinoFracture.FractureDetails:



Public Member Functions

- virtual bool [IsValid](#) ()
Returns true if the details are filled in correctly, false otherwise.

Public Attributes

- UnityEngine.Mesh **Mesh**
The mesh to fracture.
- Vector3 **MeshScale**
The scale of the mesh's game object. The meshes of fracture pieces will be scaled by this amount to allow their game object's scales to be one.
- [FractureUVScale](#) **UVScale**
Scaling algorithm used on triangles produced during the fracture.
- [FractureIssueResolution](#) **IssueResolution**
How to deal with potentially poorly generated pieces.
- bool **Asynchronous**
If true, fracturing is done on a background thread and results may not be ready by the time `FractureBuilder.Fracture()` finishes. If false, fracturing is guaranteed to be done by the time `FractureBuilder.Fracture()` finishes.
- int [InsideMaterialIndex](#)
The material / sub-mesh index that newly formed triangles should be put in.
- bool [SeparateDisjointPieces](#)
If true, a final pass will be done to separate out meshes that are not physically connected. This can only happen when the mesh has concave parts.
- [MeshValidity](#) **Validity** = [MeshValidity.Unknown](#)
Highly recommended to call

6.9.1 Detailed Description

Basic information for any sort of mesh fragmentation.

6.9.2 Member Function Documentation

6.9.2.1 IsValid()

```
virtual bool DinoFracture.FractureDetails.IsValid ( ) [virtual]
```

Returns true if the details are filled in correctly, false otherwise.

Returns

Reimplemented in [DinoFracture.ShatterDetails](#), and [DinoFracture.SliceDetails](#).

6.9.3 Member Data Documentation

6.9.3.1 InsideMaterialIndex

```
int DinoFracture.FractureDetails.InsideMaterialIndex
```

The material / sub-mesh index that newly formed triangles should be put in.

Specify -1 to put at the end of the list.

6.9.3.2 IssueResolution

```
FractureIssueResolution DinoFracture.FractureDetails.IssueResolution
```

How to deal with potentially poorly generated pieces.

Note: Any generated mesh with zero triangles is automatically removed.

6.9.3.3 SeparateDisjointPieces

```
bool DinoFracture.FractureDetails.SeparateDisjointPieces
```

If true, a final pass will be done to separate out meshes that are not physically connected. This can only happen when the mesh has concave parts.

This process can be slow. It is recommended to be off for runtime fractures unless there is a good chance of disjoint pieces.

6.10 DinoFracture.FracturedMesh Struct Reference

An individual fracture piece's geometry

Public Attributes

- `UnityEngine.Mesh` **Mesh**
The generated Unity mesh
- `FracturedMeshResultFlags` **Flags**
Additional information about the generated result.
- `Vector3` **Offset**
The offset from the origin of the original mesh to the center of this piece.
- `int` **EmptyTriangleCount**
The number of materials that have no triangles and have been removed.
- `List< bool >` **EmptyTriangles**
A true for each material from the original mesh that now have zero triangles and have been removed from the mesh.

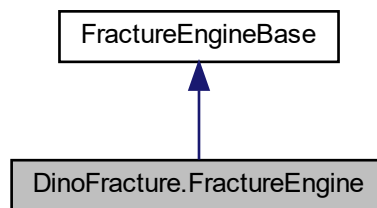
6.10.1 Detailed Description

An individual fracture piece's geometry

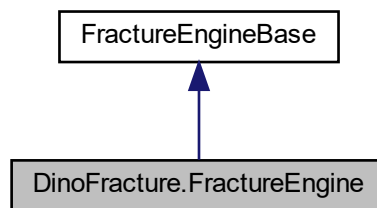
6.11 DinoFracture.FractureEngine Class Reference

This component is created on demand to manage the fracture coroutines. It is not intended to be added by the user.

Inheritance diagram for DinoFracture.FractureEngine:



Collaboration diagram for DinoFracture.FractureEngine:



Static Public Member Functions

- static [AsyncFractureResult](#) [StartFracture](#) ([FractureDetails](#) details, [FractureGeometry](#) callback, Transform piecesParent, bool transferMass, bool hideAfterFracture)
Starts a fracture operation

Protected Member Functions

- override void **Update** ()
Update

Properties

- static bool **Suspended** [get, set]
True if all further fracture operations should be a no-op.
- static bool **HasFracturesInProgress** [get]
Returns true if there are fractures currently in progress
- static int **MaxRunningFractures** [get]
The maximum number of async fractures we can process at a time. If this is set to 0 (default), an unlimited number can be run.

6.11.1 Detailed Description

This component is created on demand to manage the fracture coroutines. It is not intended to be added by the user.

6.11.2 Member Function Documentation

6.11.2.1 StartFracture()

```
static AsyncFractureResult DinoFracture.FractureEngine.StartFracture (
    FractureDetails details,
    FractureGeometry callback,
    Transform piecesParent,
    bool transferMass,
    bool hideAfterFracture ) [static]
```

Starts a fracture operation

Parameters

<i>details</i>	Fracture info
<i>callback</i>	The object to fracture
<i>piecesParent</i>	The parent of the resulting fractured pieces root object
<i>transferMass</i>	True to distribute the original object's mass to the fracture pieces; false otherwise
<i>hideAfterFracture</i>	True to hide the originating object after fracturing

Returns

6.11.3 Property Documentation

6.11.3.1 MaxRunningFractures

```
int DinoFracture.FractureEngine.MaxRunningFractures [static], [get]
```

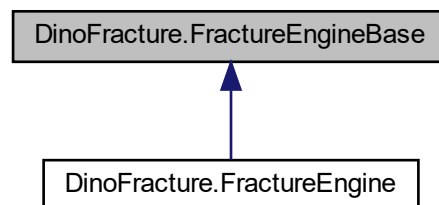
The maximum number of async fractures we can process at a time. If this is set to 0 (default), an unlimited number can be run.

NOTE: Synchronous fractures always run immediately

6.12 DinoFracture.FractureEngineBase Class Reference

Base class for the fracture engine.

Inheritance diagram for DinoFracture.FractureEngineBase:



Static Public Member Functions

- static void [ClearCachedFractureData](#) ()

During both slicing and shattering, temporary data is created and cached to greatly improve future fracture performance.

Protected Member Functions

- virtual void **Update** ()
Update
- void [RunOnMainThread](#) (Action action)
Run delegate on main thread

Properties

- static [FractureEngineBase Instance](#) [get, set]
Internal instance
- static bool [ForceSynchronousPreFractureInEditor](#) [get]
If true, pre-fracturing in the editor will always be synchronous.

6.12.1 Detailed Description

Base class for the fracture engine.

6.12.2 Member Function Documentation

6.12.2.1 ClearCachedFractureData()

```
static void DinoFracture.FractureEngineBase.ClearCachedFractureData ( ) [static]
```

During both slicing and shattering, temporary data is created and cached to greatly improve future fracture performance.

Calling this method releases that temporary data back to be reclaimed by the GC.

Calling this method may cause a large GC spike soon after. It is recommended to call this during 'downtime' in the game, such as during level load.

6.12.2.2 RunOnMainThread()

```
void DinoFracture.FractureEngineBase.RunOnMainThread (
    Action action ) [protected]
```

Run delegate on main thread

Parameters

<i>action</i>	
---------------	--

6.12.3 Property Documentation

6.12.3.1 ForceSynchronousPreFractureInEditor

```
bool DinoFracture.FractureEngineBase.ForceSynchronousPreFractureInEditor [static], [get]
```

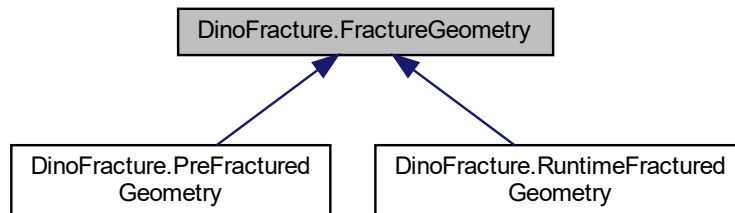
If true, pre-fracturing in the editor will always be synchronous.

This is mainly used for debugging purposes.

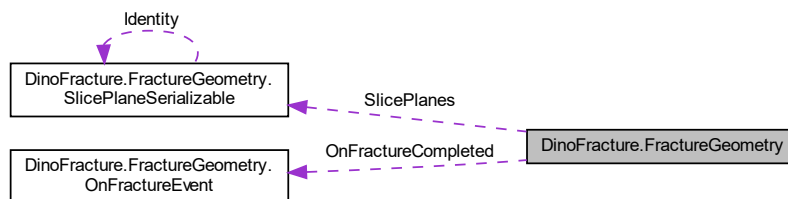
6.13 DinoFracture.FractureGeometry Class Reference

This is the base class for the `PreFractureGeometry` and `RuntimeFractureGeometry` components. As such, it is not intended to be directly added to any game object even though fracture initiator components rely on it.

Inheritance diagram for `DinoFracture.FractureGeometry`:



Collaboration diagram for `DinoFracture.FractureGeometry`:



Classes

- class [OnFractureEvent](#)
OnFracture() Unity event wrapper
- struct [SlicePlaneSerializable](#)
Unity cannot handle the serializable attribute on types defined in dlls. So, we have to duplicate the [SlicePlane](#) structure here in order to save it.

Public Member Functions

- void [FractureAndForget](#) ()
Initiate a fracture at the origin and does not return a handle to the async operation.
- void [FractureAndForget](#) (Vector3 localPos)
Initiate a fracture at the specified position relative to this object and does not return a handle to the async operation.
- [AsyncFractureResult Fracture](#) ()
Initiate a fracture at the origin

- [AsyncFractureResult Fracture](#) (Vector3 localPos)
Initiate a fracture at the specified position relative to this object.
- [MeshTopologyError CheckMeshValidity](#) ()
This is called automatically when viewing the component in the inspector. However, it should be called whenever the mesh changes through other means.
- void [ForceValidGeometry](#) ()
This will force the geometry to pass all validity checks.
- List< [EdgeError](#) > [GetMeshEdgeErrors](#) ()
Returns any bad mesh edges. Used by the editor script for debugging.

Static Public Member Functions

- static [SlicePlaneSerializable CreateSlicePlane](#) (Plane worldPlane, Transform targetGameObject)
This will create a valid slice plane for slicing a mesh.

Public Attributes

- Material **InsideMaterial**
The material assigned to the "inside" triangles of the fracture pieces. These are the triangles that [DinoFracture](#) creates. The surface triangles of the original mesh retain their materials.
- bool [OptimizeMaterialUsage](#) = true
If true, newly generated triangles using the "InsideMaterial" will attempt to be part of the same existing material in the mesh.
- GameObject **FractureTemplate**
This game object will be cloned for each fracture piece. It is required to have a MeshFilter component. If a MeshCollider component is added, it will be assigned the fracture mesh.
- Transform **PiecesParent**
The parent of the generated pieces. Each fracture produces a root object with fracture pieces (clones of [FractureTemplate](#)) as children. The root object is parented to PiecesParent.
- [FractureType](#) **FractureType** = [FractureType.Shatter](#)
The type of fracture to produce when [Fracture\(\)](#) is called.
- [SlicePlaneSerializable\[\]](#) [SlicePlanes](#)
The planes to use when slicing the mesh. Not used when fracturing into pieces.
- int **NumFracturePieces** = 5
The number of fracture pieces generated per iteration. Fault lines are spread evenly around the fracture point. The number of total pieces generated is NumFracturePieces ^ NumIterations.
- int [NumIterations](#) = 2
The number of passes of fracturing. Using lower piece count with a higher iteration count is computationally faster than a higher piece count with a lower iteration count. Ex: 5 pieces with 2 iterations is faster than 25 pieces and 1 iteration. The downside to using more iterations is fractures can become less uniform. In general, keep this number below 4. The number of total pieces generated is NumFracturePieces ^ NumIterations.
- bool [EvenlySizedPieces](#)
If true, the engine will attempt to make all the randomly generated pieces roughly the same size. This adds a little processing time to the fracture.
- int [NumGenerations](#) = 1
To allow for fracture pieces to be further fractured, the FractureTemplate should have a [FractureGeometry](#) component. NumGenerations dictates how many times the geometry can be re-fractured. The count is decremented and passed on to the component in each generated piece. Ex: A value of 2 means this piece can be fractured and each generated piece can be fractured. The second generation of fractures cannot be fractured further.
- float **FractureRadius**

A value between 0 and 1 that indicates how clustered the fracture lines are. A value of 0 or 1 means fractures are evenly distributed across the mesh. A value between means they are clustered within a percentage of the mesh bounds. Ex: a value of 0.3 means fractures are clustered around the fracture point in a volume 30% the size of the mesh. Pre-fracture geometry typically has this value set to 0 or 1 because there isn't always a pre-determined point of fracture.

- [FractureUVScale](#) **UVScale** = [FractureUVScale.Piece](#)

If set to `EntireMesh`, the UV map for each inside triangle will be mapped to a box the size of the original mesh. If set to `piece`, inside triangles will be mapped to a box the size of the individual fracture piece.

- bool **DistributeMass** = true

If true and both this game object and the `FractureTemplate` have a `RigidBody` component, each fracture piece will have a mass set to a value proportional to its volume. That is, the density of the fracture piece will equal the density of the original mesh. If false, the mass property goes untouched.

- bool [SeparateDisjointPieces](#) = false

If true, a final pass will be done to separate out meshes that are not physically connected. This can only happen when the mesh has concave parts.

- int **RandomSeed** = 0

The random seed to use when initiating the fracture. If set to zero, then the system clock will be used to create a random seed.

- [OnFractureEvent](#) **OnFractureCompleted**

Unity event that fires whenever a fracture on this object completes

- bool **SkipMeshCleaning** = false

Not recommended to be set. But if set to true, no mesh cleaning will occur during the fracture process. This has no effect if the mesh is already clean.

Properties

- string **Uniqueld** [get]
Used internally to uniquely identify this object
- [MeshValidity](#) **MeshValidity** [get]
Used to determine if the mesh is of known good quality to fracture.
- bool **IsProcessingFracture** [get]
Are we in the middle of computing a fracture for this object?

6.13.1 Detailed Description

This is the base class for the `PreFractureGeometry` and `RuntimeFractureGeometry` components. As such, it is not intended to be directly added to any game object even though fracture initiator components rely on it.

6.13.2 Member Function Documentation

6.13.2.1 CreateSlicePlane()

```
static SlicePlaneSerializable DinoFracture.FractureGeometry.CreateSlicePlane (
    Plane worldPlane,
    Transform targetGameObject ) [static]
```

This will create a valid slice plane for slicing a mesh.

While valid, it is not intended to be displayed in the editor and is meant for runtime use.

6.13.2.2 ForceValidGeometry()

```
void DinoFracture.FractureGeometry.ForceValidGeometry ( )
```

This will force the geometry to pass all validity checks.

This method can be useful to call in very custom scenarios of fracturing through script when you know the mesh you are passing in is valid.

6.13.2.3 Fracture() [1/2]

```
AsyncFractureResult DinoFracture.FractureGeometry.Fracture ( )
```

Initiate a fracture at the origin

Returns

6.13.2.4 Fracture() [2/2]

```
AsyncFractureResult DinoFracture.FractureGeometry.Fracture (
    Vector3 localPos )
```

Initiate a fracture at the specified position relative to this object.

Parameters

<i>localPos</i>	
-----------------	--

Returns

6.13.2.5 FractureAndForget() [1/2]

```
void DinoFracture.FractureGeometry.FractureAndForget ( )
```

Initiate a fracture at the origin and does not return a handle to the async operation.

The OnFracture() callback will still fire. This method is compatible with Unity events.

6.13.2.6 FractureAndForget() [2/2]

```
void DinoFracture.FractureGeometry.FractureAndForget (
    Vector3 localPos )
```

Initiate a fracture at the specified position relative to this object and does not return a handle to the async operation.

The OnFracture() callback will still fire. This method is compatible with Unity events.

6.13.3 Member Data Documentation

6.13.3.1 EvenlySizedPieces

```
bool DinoFracture.FractureGeometry.EvenlySizedPieces
```

If true, the engine will attempt to make all the randomly generated pieces roughly the same size. This adds a little processing time to the fracture.

Do not set this to true if FractureRadius > 0.

6.13.3.2 NumGenerations

```
int DinoFracture.FractureGeometry.NumGenerations = 1
```

To allow for fracture pieces to be further fractured, the FractureTemplate should have a [FractureGeometry](#) component. NumGenerations dictates how many times the geometry can be re-fractured. The count is decremented and passed on to the component in each generated piece. Ex: A value of 2 means this piece can be fractured and each generated piece can be fractured. The second generation of fractures cannot be fractured further.

Specify a negative value on the main piece to allow for infinite repeated fractures

6.13.3.3 NumIterations

```
int DinoFracture.FractureGeometry.NumIterations = 2
```

The number of passes of fracturing. Using lower piece count with a higher iteration count is computationally faster than a higher piece count with a lower iteration count. Ex: 5 pieces with 2 iterations is faster than 25 pieces and 1 iteration. The downside to using more iterations is fractures can become less uniform. In general, keep this number below 4. The number of total pieces generated is NumFracturePieces ^ NumIterations.

It is recommended you use an iteration count of 1 when $0 < \text{FractureRadius} < 1$.

6.13.3.4 OptimizeMaterialUsage

```
bool DinoFracture.FractureGeometry.OptimizeMaterialUsage = true
```

If true, newly generated triangles using the "InsideMaterial" will attempt to be part of the same existing material in the mesh.

If false, newly generated triangles will always be put in a new material placed after all *used* existing materials. Additionally, new materials will continue to be appended upon further fractures.

6.13.3.5 SeparateDisjointPieces

```
bool DinoFracture.FractureGeometry.SeparateDisjointPieces = false
```

If true, a final pass will be done to separate out meshes that are not physically connected. This can only happen when the mesh has concave parts.

This process can be slow. It is recommended to be off for runtime fractures unless there is a good chance of disjoint pieces.

6.13.3.6 SlicePlanes

```
SlicePlaneSerializable [] DinoFracture.FractureGeometry.SlicePlanes
```

The planes to use when slicing the mesh. Not used when fracturing into pieces.

Each slice plane must be in local space. Use [FractureGeometry.CreateSlicePlane\(\)](#) to create a compatible local space plane from a Unity world space plane.

6.14 DinoFracture.FractureOnClick Class Reference

Casts a simple mouse ray on left click and calls `Fracture()` on the hit collider game object.

Inherits `MonoBehaviour`.

6.14.1 Detailed Description

Casts a simple mouse ray on left click and calls `Fracture()` on the hit collider game object.

6.15 DinoFracture.FractureOnCollision Class Reference

This component will cause a fracture to happen at the point of impact.

Inherits `MonoBehaviour`.

Public Attributes

- float **ForceThreshold**
The minimum amount of force required to fracture this object. Set to 0 to have any amount of force cause the fracture.
- float **ForceFalloffRadius** = 1.0f
Falloff radius for transferring the force of the impact to the resulting pieces. Any piece outside of this falloff from the point of impact will have no additional impulse set on it.
- bool **AdjustForKinematic** = true
If true and this is a kinematic body, an impulse will be applied to the colliding body to counter the effects of hitting a kinematic body. If false and this is a kinematic body, the colliding body will bounce off as if this were an unmovable wall.

6.15.1 Detailed Description

This component will cause a fracture to happen at the point of impact.

6.16 DinoFracture.FractureOnInput Class Reference

Apply this on the fracturing game object. When the specified key is pressed, the object will fracture.

Inherits MonoBehaviour.

6.16.1 Detailed Description

Apply this on the fracturing game object. When the specified key is pressed, the object will fracture.

6.17 DinoFracture.FractureResult Class Reference

The result of a fracture.

Public Member Functions

- IReadOnlyList< [FracturedMesh](#) > [GetMeshes](#) ()
Returns a list of pieces produced by the fracture

Properties

- Bounds **EntireMeshBounds** [get]
Bounds of the original mesh, in local space

6.17.1 Detailed Description

The result of a fracture.

6.17.2 Member Function Documentation

6.17.2.1 GetMeshes()

```
ICollection< FracturedMesh > DinoFracture.FractureResult.GetMeshes ( )
```

Returns a list of pieces produced by the fracture

Returns

6.18 DinoFracture.GlueEdgeOnFracture Class Reference

If the fracture pieces intersects with a specified trigger when created, the rigid body is destroyed and the piece becomes static. Otherwise, the piece will turn on gravity. It's best used if the FractureTemplate's rigid body is set to not use gravity initially.

Inherits MonoBehaviour.

Public Attributes

- string **CollisionTag** = ""
The piece will be glued if it intersects a trigger with this collision tag. Set to empty to allow any trigger to glue the piece.

6.18.1 Detailed Description

If the fracture pieces intersects with a specified trigger when created, the rigid body is destroyed and the piece becomes static. Otherwise, the piece will turn on gravity. It's best used if the FractureTemplate's rigid body is set to not use gravity initially.

6.19 DinoFracture.NotifyOnFracture Class Reference

When added to the same game object as the [FractureGeometry](#), this script can be used to notify external game objects of this object's fracture completion. The external objects need a script with the "OnFracture" callback method.

Inherits MonoBehaviour.

Public Attributes

- GameObject[] **GameObjects** = new GameObject[1]
The array of game objects to notify. They do not need to be in this object's tree.

6.19.1 Detailed Description

When added to the same game object as the [FractureGeometry](#), this script can be used to notify external game objects of this object's fracture completion. The external objects need a script with the "OnFracture" callback method.

6.20 DinoFracture.FractureGeometry.OnFractureEvent Class Reference

OnFracture() Unity event wrapper

Inherits `UnityEngine.Events.UnityEvent< OnFractureEventArgs >`.

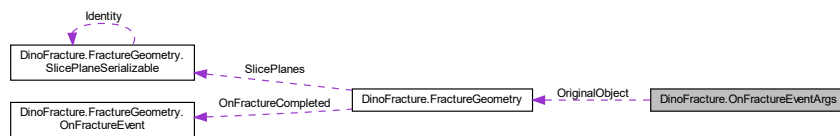
6.20.1 Detailed Description

OnFracture() Unity event wrapper

6.21 DinoFracture.OnFractureEventArgs Class Reference

Argument passed to OnFracture message

Collaboration diagram for `DinoFracture.OnFractureEventArgs`:



Public Member Functions

- `IEnumerable< UnityEngine.Mesh > GetMeshes ()`
Returns an enumerable of just the generated Unity meshes

Public Attributes

- `FractureGeometry OriginalObject`
The object that fractured.
- Bounds `OriginalMeshBounds`
The bounds of the original mesh
- GameObject `FracturePiecesRootObject`
The root of the pieces of the resulting fracture.

6.21.1 Detailed Description

Argument passed to OnFracture message

6.21.2 Member Function Documentation

6.21.2.1 GetMeshes()

```
IEnumerable< UnityEngine.Mesh > DinoFracture.OnFractureEventArgs.GetMeshes ( )
```

Returns an enumerable of just the generated Unity meshes

Returns

6.22 DinoFracture.PlaySoundOnFracture Class Reference

An object with this component will play the audio source when fractured.

Inherits MonoBehaviour.

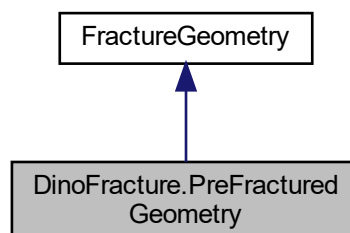
6.22.1 Detailed Description

An object with this component will play the audio source when fractured.

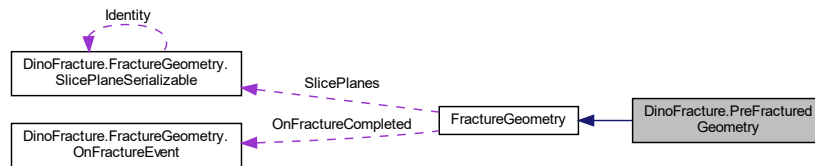
6.23 DinoFracture.PreFracturedGeometry Class Reference

Apply this component to any game object you wish to pre-fracture. Pre-fracturing is a way of baking fracture pieces into the scene. Each time the object is fractured, the same set of pieces will activate. This is very useful when creating a large number of pieces or high poly meshes, which would be too slow to create at runtime. The pieces will be in the scene as a disabled root object with piece children. When the object is fractured, those pieces will activate.

Inheritance diagram for DinoFracture.PreFracturedGeometry:



Collaboration diagram for DinoFracture.PreFracturedGeometry:



Public Member Functions

- void **Prime** ()

Primes the pre-fractured pieces when the game starts by activating them and then deactivating them. This avoids a large delay on fracture if there are a lot of rigid bodies.

Public Attributes

- GameObject [GeneratedPieces](#)

A reference to the root of the pre-fractured pieces. This is not normally set manually. Instead, you press the "Create Fractures" button in the inspector window to generate the fracture immediately.

- Bounds **EntireMeshBounds**

The encapsulating bounds of the entire set of pieces. In local space.

Additional Inherited Members

6.23.1 Detailed Description

Apply this component to any game object you wish to pre-fracture. Pre-fracturing is a way of baking fracture pieces into the scene. Each time the object is fractured, the same set of pieces will activate. This is very useful when creating a large number of pieces or high poly meshes, which would be too slow to create at runtime. The pieces will be in the scene as a disabled root object with piece children. When the object is fractured, those pieces will activate.

6.23.2 Member Data Documentation

6.23.2.1 GeneratedPieces

GameObject DinoFracture.PreFracturedGeometry.GeneratedPieces

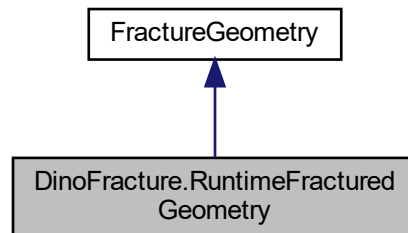
A reference to the root of the pre-fractured pieces. This is not normally set manually. Instead, you press the "Create Fractures" button in the inspector window to generate the fracture immediately.

The "Create Fractures" button is only intended to be used in edit mode; not game mode.

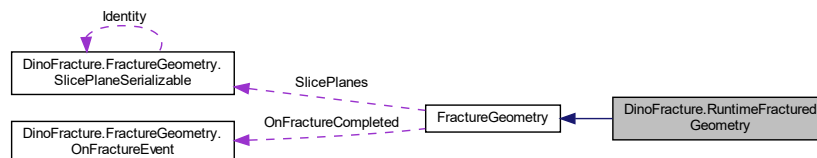
6.24 DinoFracture.RuntimeFracturedGeometry Class Reference

Apply this component to any game object you wish to fracture while running in game mode. Runtime fractures will produce a unique set of pieces with each fracture. However, this is at the cost of computational time. It is recommended that both the piece count and poly count are kept low. This component is most effective when FractureRadius is set to a value in-between 0 and 1.

Inheritance diagram for DinoFracture.RuntimeFracturedGeometry:



Collaboration diagram for DinoFracture.RuntimeFracturedGeometry:



Public Attributes

- bool `Asynchronous` = true

If true, the fracture operation is performed on a background thread and may not be finished by the time the fracture call returns. A couple of frames can go by from the time of the fracture to when the pieces are ready. If this is false, the fracture will guaranteed be complete by the end of the call, but the game will be paused while the fractures are being created.

Additional Inherited Members

6.24.1 Detailed Description

Apply this component to any game object you wish to fracture while running in game mode. Runtime fractures will produce a unique set of pieces with each fracture. However, this is at the cost of computational time. It is recommended that both the piece count and poly count are kept low. This component is most effective when FractureRadius is set to a value in-between 0 and 1.

6.24.2 Member Data Documentation

6.24.2.1 Asynchronous

```
bool DinoFracture.RuntimeFracturedGeometry.Asynchronous = true
```

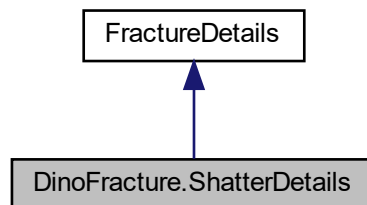
If true, the fracture operation is performed on a background thread and may not be finished by the time the fracture call returns. A couple of frames can go by from the time of the fracture to when the pieces are ready. If this is false, the fracture will guaranteed be complete by the end of the call, but the game will be paused while the fractures are being created.

It is recommended to set asynchronous to true whenever possible.

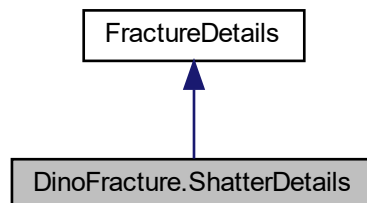
6.25 DinoFracture.ShatterDetails Class Reference

Required information needed by the engine to produce a fracture

Inheritance diagram for DinoFracture.ShatterDetails:



Collaboration diagram for DinoFracture.ShatterDetails:



Public Member Functions

- override bool [IsValid](#) ()
Returns true if the details are filled in correctly, false otherwise.

Public Attributes

- int **NumPieces**
The number of new pieces to produce per iteration. The total number of pieces produced by this fracture will be $\text{NumPieces} \wedge \text{NumIterations}$.
- int **NumIterations**
The number of fracture iterations. The total number of pieces produced by this fracture will be $\text{NumPieces} \wedge \text{NumIterations}$.
- bool [EvenlySizedPieces](#)
If true, the engine will attempt to make all the randomly generated pieces roughly the same size. This adds a little processing time to the fracture.
- Vector3 **FractureCenter**
The center of the fracture.
- float **FractureRadius**
A value between 0 and 1 that indicates how clustered fracture lines will be. A value of 0 or 1 evenly distributes fracture lines across the entire mesh. A value in between 0 and 1 will distribute lines around FractureCenter in a radius that is a fraction of the size of the mesh.
- int **RandomSeed**
The random seed to use when initiating the fracture. If set to zero, then the system clock will be used to create a random seed.

6.25.1 Detailed Description

Required information needed by the engine to produce a fracture

6.25.2 Member Function Documentation

6.25.2.1 IsValid()

```
override bool DinoFracture.ShatterDetails.IsValid ( ) [virtual]
```

Returns true if the details are filled in correctly, false otherwise.

Returns

Reimplemented from [DinoFracture.FractureDetails](#).

6.25.3 Member Data Documentation

6.25.3.1 EvenlySizedPieces

```
bool DinoFracture.ShatterDetails.EvenlySizedPieces
```

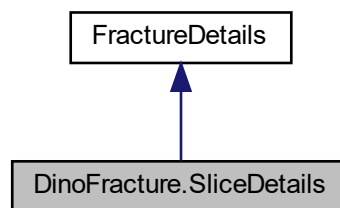
If true, the engine will attempt to make all the randomly generated pieces roughly the same size. This adds a little processing time to the fracture.

Do not set this to true if FractureRadius > 0.

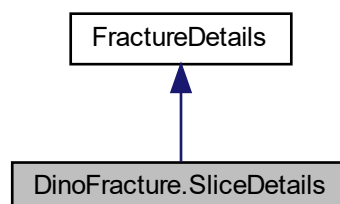
6.26 DinoFracture.SliceDetails Class Reference

Required information needed by the engine to slice a mesh

Inheritance diagram for DinoFracture.SliceDetails:



Collaboration diagram for DinoFracture.SliceDetails:



Public Member Functions

- override bool [IsValid](#) ()
Returns true if the details are filled in correctly, false otherwise.

Public Attributes

- readonly List< [SlicePlane](#) > **SlicingPlanes** = new List<[SlicePlane](#)>()
User defined slicing planes.

6.26.1 Detailed Description

Required information needed by the engine to slice a mesh

6.26.2 Member Function Documentation

6.26.2.1 IsValid()

```
override bool DinoFracture.SliceDetails.IsValid ( ) [virtual]
```

Returns true if the details are filled in correctly, false otherwise.

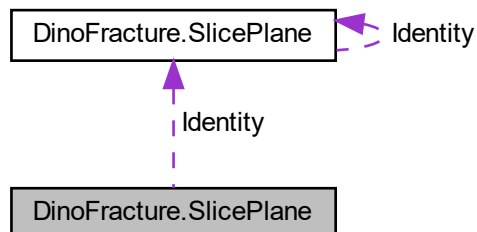
Returns

Reimplemented from [DinoFracture.FractureDetails](#).

6.27 DinoFracture.SlicePlane Struct Reference

Defines a plane that slices the mesh in half.

Collaboration diagram for DinoFracture.SlicePlane:



Public Member Functions

- Plane **ToPlane** ()
Converts this object to a Unity plane.

Public Attributes

- Vector3 **Position**
Local space position of the plane
- Quaternion **Rotation**
Local space rotation of the plane.
- float **Scale**
Scale of the plane.

Static Public Attributes

- static readonly **SlicePlane Identity**
Default, "identity" plane

6.27.1 Detailed Description

Defines a plane that slices the mesh in half.

Values are in the mesh's local space.

6.27.2 Member Data Documentation

6.27.2.1 Rotation

```
Quaternion DinoFracture.SlicePlane.Rotation
```

Local space rotation of the plane.

Z dir is the plane's normal.

6.27.2.2 Scale

```
float DinoFracture.SlicePlane.Scale
```

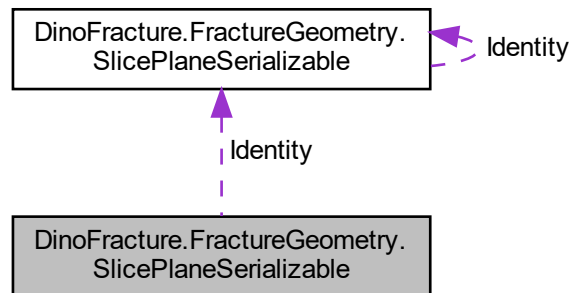
Scale of the plane.

This is only used to display it in the editor. When slicing, the plane will be treated as infinite in size.

6.28 DinoFracture.FractureGeometry.SlicePlaneSerializable Struct Reference

Unity cannot handle the serializable attribute on types defined in dlls. So, we have to duplicate the [SlicePlane](#) structure here in order to save it.

Collaboration diagram for DinoFracture.FractureGeometry.SlicePlaneSerializable:



Public Member Functions

- [SlicePlane ToSlicePlane](#) ()
Converts this serialization helper to a normal slice plane

6.28.1 Detailed Description

Unity cannot handle the serializable attribute on types defined in dlls. So, we have to duplicate the [SlicePlane](#) structure here in order to save it.

6.28.2 Member Function Documentation

6.28.2.1 ToSlicePlane()

[SlicePlane](#) DinoFracture.FractureGeometry.SlicePlaneSerializable.ToSlicePlane ()

Converts this serialization helper to a normal slice plane

Returns

6.29 DinoFracture.TransferJointsOnFracture Class Reference

When this object is fractured, the joint component on the object will be copied to this piece if this piece is sufficiently close to the joint position. Without this component, joints are broken after fracturing.

Inherits MonoBehaviour.

Public Attributes

- Transform **IncomingJointsSearchRoot**
The tree to crawl in search for joints of other objects that need to be transferred to this joint. This search root should be as scoped as possible.
- float **DistanceTolerance** = 0.05f
How close this object must be to the joint in order to transfer. The larger the number, the more pieces will have joints transferred.

6.29.1 Detailed Description

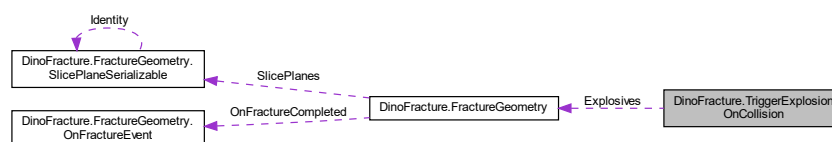
When this object is fractured, the joint component on the object will be copied to this piece if this piece is sufficiently close to the joint position. Without this component, joints are broken after fracturing.

6.30 DinoFracture.TriggerExplosionOnCollision Class Reference

Triggers a fracture + explosion when this game object is collided with.

Inherits MonoBehaviour.

Collaboration diagram for DinoFracture.TriggerExplosionOnCollision:



Public Attributes

- [FractureGeometry\[\]](#) **Explosives**
List of explosions to trigger
- float **Force**
The force behind the explosions
- float **Radius**
The radius of the explosions

6.30.1 Detailed Description

Triggers a fracture + explosion when this game object is collided with.

This script does not need to be applied on a fracturing game object.

Index

- Asynchronous
 - DinoFracture.RuntimeFracturedGeometry, [39](#)
- ClearCachedFractureData
 - DinoFracture.FractureEngineBase, [26](#)
- CloseVertices
 - DinoFracture, [12](#)
- CreateSlicePlane
 - DinoFracture.FractureGeometry, [29](#)
- DegenerateTriangles
 - DinoFracture, [12](#)
- DinoFracture, [9](#)
 - CloseVertices, [12](#)
 - DegenerateTriangles, [12](#)
 - DisableGameObject, [11](#)
 - EntireMesh, [12](#)
 - FanFaces, [12](#)
 - FracturedMeshResultFlags, [11](#)
 - FractureIssueResolution, [11](#)
 - FractureType, [11](#)
 - FractureUVScale, [11](#)
 - MeshTopologyError, [12](#)
 - MeshValidity, [12](#)
 - NeedsCleaning, [13](#)
 - NoAction, [11](#)
 - NoIssues, [11](#)
 - None, [12](#)
 - OpenFaces, [12](#)
 - Piece, [12](#)
 - ReplaceMeshCollider, [11](#)
 - Shatter, [11](#)
 - Slice, [11](#)
 - SmallVertexCount, [11](#)
 - Unknown, [12](#)
 - Unrecoverable, [13](#)
 - Valid, [12](#)
- DinoFracture.AsyncFractureOperation, [15](#)
 - Wait, [16](#)
- DinoFracture.AsyncFractureResult, [16](#)
- DinoFracture.AsyncShatterOperation, [17](#)
- DinoFracture.AsyncSliceOperation, [18](#)
- DinoFracture.CleanupMeshOnDestroy, [19](#)
- DinoFracture.DisableObjectsOnFracture, [19](#)
- DinoFracture.EdgeError, [19](#)
- DinoFracture.FractureDetails, [20](#)
 - InsideMaterialIndex, [21](#)
 - IssueResolution, [22](#)
 - IsValid, [21](#)
 - SeparateDisjointPieces, [22](#)
- DinoFracture.FracturedMesh, [22](#)
- DinoFracture.FractureEngine, [23](#)
 - MaxRunningFractures, [24](#)
 - StartFracture, [24](#)
- DinoFracture.FractureEngineBase, [25](#)
 - ClearCachedFractureData, [26](#)
 - ForceSynchronousPreFractureInEditor, [26](#)
 - RunOnMainThread, [26](#)
- DinoFracture.FractureGeometry, [27](#)
 - CreateSlicePlane, [29](#)
 - EvenlySizedPieces, [31](#)
 - ForceValidGeometry, [29](#)
 - Fracture, [30](#)
 - FractureAndForget, [30](#)
 - NumGenerations, [31](#)
 - NumIterations, [31](#)
 - OptimizeMaterialUsage, [31](#)
 - SeparateDisjointPieces, [32](#)
 - SlicePlanes, [32](#)
- DinoFracture.FractureGeometry.OnFractureEvent, [35](#)
- DinoFracture.FractureGeometry.SlicePlaneSerializable, [44](#)
 - ToSlicePlane, [44](#)
- DinoFracture.FractureOnClick, [32](#)
- DinoFracture.FractureOnCollision, [32](#)
- DinoFracture.FractureOnInput, [33](#)
- DinoFracture.FractureResult, [33](#)
 - GetMeshes, [34](#)
- DinoFracture.GlueEdgeOnFracture, [34](#)
- DinoFracture.Internal, [13](#)
- DinoFracture.Internal.DestroyOnAudioFinish, [19](#)
- DinoFracture.NotifyOnFracture, [34](#)
- DinoFracture.OnFractureEventArgs, [35](#)
 - GetMeshes, [36](#)
- DinoFracture.PlaySoundOnFracture, [36](#)
- DinoFracture.PreFracturedGeometry, [36](#)
 - GeneratedPieces, [37](#)
- DinoFracture.RuntimeFracturedGeometry, [38](#)
 - Asynchronous, [39](#)
- DinoFracture.ShatterDetails, [39](#)
 - EvenlySizedPieces, [41](#)
 - IsValid, [40](#)
- DinoFracture.SliceDetails, [41](#)
 - IsValid, [42](#)
- DinoFracture.SlicePlane, [42](#)
 - Rotation, [43](#)
 - Scale, [43](#)
- DinoFracture.TransferJointsOnFracture, [45](#)
- DinoFracture.TriggerExplosionOnCollision, [45](#)

- DisableGameObject
 - DinoFracture, [11](#)
- EntireMesh
 - DinoFracture, [12](#)
- EvenlySizedPieces
 - DinoFracture.FractureGeometry, [31](#)
 - DinoFracture.ShatterDetails, [41](#)
- FanFaces
 - DinoFracture, [12](#)
- ForceSynchronousPreFractureInEditor
 - DinoFracture.FractureEngineBase, [26](#)
- ForceValidGeometry
 - DinoFracture.FractureGeometry, [29](#)
- Fracture
 - DinoFracture.FractureGeometry, [30](#)
- FractureAndForget
 - DinoFracture.FractureGeometry, [30](#)
- FracturedMeshResultFlags
 - DinoFracture, [11](#)
- FractureIssueResolution
 - DinoFracture, [11](#)
- FractureType
 - DinoFracture, [11](#)
- FractureUVScale
 - DinoFracture, [11](#)
- GeneratedPieces
 - DinoFracture.PreFracturedGeometry, [37](#)
- GetMeshes
 - DinoFracture.FractureResult, [34](#)
 - DinoFracture.OnFractureEventArgs, [36](#)
- InsideMaterialIndex
 - DinoFracture.FractureDetails, [21](#)
- IssueResolution
 - DinoFracture.FractureDetails, [22](#)
- IsValid
 - DinoFracture.FractureDetails, [21](#)
 - DinoFracture.ShatterDetails, [40](#)
 - DinoFracture.SliceDetails, [42](#)
- MaxRunningFractures
 - DinoFracture.FractureEngine, [24](#)
- MeshTopologyError
 - DinoFracture, [12](#)
- MeshValidity
 - DinoFracture, [12](#)
- NeedsCleaning
 - DinoFracture, [13](#)
- NoAction
 - DinoFracture, [11](#)
- NoIssues
 - DinoFracture, [11](#)
- None
 - DinoFracture, [12](#)
- NumGenerations
 - DinoFracture.FractureGeometry, [31](#)
- NumIterations
 - DinoFracture.FractureGeometry, [31](#)
- OpenFaces
 - DinoFracture, [12](#)
- OptimizeMaterialUsage
 - DinoFracture.FractureGeometry, [31](#)
- Piece
 - DinoFracture, [12](#)
- ReplaceMeshCollider
 - DinoFracture, [11](#)
- Rotation
 - DinoFracture.SlicePlane, [43](#)
- RunOnMainThread
 - DinoFracture.FractureEngineBase, [26](#)
- Scale
 - DinoFracture.SlicePlane, [43](#)
- SeparateDisjointPieces
 - DinoFracture.FractureDetails, [22](#)
 - DinoFracture.FractureGeometry, [32](#)
- Shatter
 - DinoFracture, [11](#)
- Slice
 - DinoFracture, [11](#)
- SlicePlanes
 - DinoFracture.FractureGeometry, [32](#)
- SmallVertexCount
 - DinoFracture, [11](#)
- StartFracture
 - DinoFracture.FractureEngine, [24](#)
- ToSlicePlane
 - DinoFracture.FractureGeometry.SlicePlaneSerializable, [44](#)
- Unknown
 - DinoFracture, [12](#)
- Unrecoverable
 - DinoFracture, [13](#)
- Valid
 - DinoFracture, [12](#)
- Wait
 - DinoFracture.AsyncFractureOperation, [16](#)